Contents

1 Exact versus approximate evaluation of ${}_6F_5$ conservative-matrix-field chains on the Apple M5 Max: a reproducible latency-and-throughput study 1

1.1 Abstract . . . 1
1.2 1. Introduction . . . 2
1.3 2. Background . . . 3
1.3.1 2.1 Projective matrix-chain limit . . . 3
1.3.2 2.2 Residue number systems and Montgomery arithmetic . . . 3
1.3.3 2.3 Double-float (df64) arithmetic . . . 3
1.4 3. Methods . . . 3
1.4.1 3.1 Corpus . . . 3
1.4.2 3.2 Back-ends . . . 4
1.4.3 3.3 Timing methodology and throughput definitions . . . 4
1.4.4 3.4 Hardware and software . . . 4
1.5 4. Results . . . 5
1.5.1 4.1 Correctness across three workloads . . . 5
1.5.2 4.2 Single-chain latency (`sparse_x1`) . . . 5
1.5.3 4.3 Throughput: the fair comparison . . . 6
1.5.4 4.4 Robustness across workloads . . . 6
1.5.5 4.5 Bit growth and the exact-arithmetic crossover . . . 8
1.5.6 4.6 Unique-trajectory production throughput . . . 8
1.6 5. Discussion . . . 9
1.7 6. Reproducibility . . . 10
1.8 7. Limitations and future work . . . 10
1.9 Data Availability Statement . . . 10
1.10 Author Contributions . . . 10
1.11 Funding . . . 11
1.12 Conflict of Interest . . . 11
1.13 Generative AI Disclosure . . . 11
1.14 Acknowledgements . . . 11
1.15 References . . . 11

1 Exact versus approximate evaluation of ${}_6F_5$ conservative-matrix-field chains on the Apple M5 Max: a reproducible latency-and-throughput study

David Vesterlund[1,*]

[1]Independent researcher. ORCID: 0009-0000-6455-1141.

[*]**Correspondence:** David Vesterlund (corresponding author).

Keywords: residue number system; exact arithmetic; Apple Silicon; Metal; matrix multiplication; experimental mathematics; conservative matrix fields; high-performance computing.

Submitted to Frontiers in High Performance Computing, section Benchmarking (article type: Original Research).

1.1 Abstract

Experimental-mathematics engines in the Ramanujan Machine family reduce the discovery of new closed forms for mathematical constants to a single hot loop: multiply a long chain of small integer matrices and

inspect the projective limit. On a rank-6 (${}_6F_5$) conservative matrix field (CMF), the accumulated 6×6 product reaches 9546 bits after only 1000 unit-axis steps, so the choice of arithmetic — exact big-integer, residue-number-system (RNS), or fast but lossy floating point — directly determines both throughput and scientific validity. We present a fully reproducible benchmark that fixes SHA-256-pinned operator corpora derived from the corrected Ramanujan Dreams ${}_6F_5$ walk and times exact and approximate back-ends against an exact reference on an Apple M5 Max laptop: Python `int`, SymPy, python-flint/FLINT, GMP, Boost.Multiprecision, a threaded Montgomery-RNS CPU engine, a Metal RNS GPU kernel, Apple Accelerate BLAS, and a Metal double-float (df64) GPU kernel. Every exact back-end reproduces the reference digest across three workloads (`sparse_x1`, `x6`, `dense_pos`); every floating-point back-end is scored by projective error. Crucially, we compare *like with like*: GPU batch throughput is set against CPU **trajectory-parallel** throughput (many independent chains across all 18 cores), and we separate RNS **modular-kernel** throughput from **end-to-end exact** throughput that includes Chinese-Remainder (CRT) reconstruction. Three regimes emerge. (i) For a single exact chain, big-integer GMP is fastest (11.8 ms). (ii) For exact *throughput*, the RNS GPU modular kernel peaks near 4.7k trajectories/GPU-s, but once per-trajectory CRT is required the exact RNS path is reconstruction-bound ($\$ 0.2k/s), $ $below trajectory - parallel GMP($ \$0.7k/s) — so the GPU wins on exact arithmetic only when reconstruction is amortised or work stays in residue space. (iii) For approximate screening, the df64 GPU (up to $ $17ktraj/s) $ only matches the 18 - core Accelerate CPU($ \$17k/s) at large batch, because a 6×6 multiply is too small to amortise GPU launch. A bit-growth study across $N \leq 2000$ (K from 10 to 703 primes) locates the exact-arithmetic crossover as a function of integer size. We give the RNS modulus-capacity certificate, distinguish *a posteriori* from *a priori* capacity, release two figures, and publish all code, data, dumps, and provenance under a permanent DOI.

1.2 1. Introduction

The Ramanujan Machine (Raayoni et al. 2021) and its conservative-matrix-field generalisation (Elimelech et al. 2024; Weinbaum et al. 2025) search vast spaces of matrix-valued recurrences for polynomial continued fractions that converge to fundamental constants. In the CMF formulation, a "walk" is an ordered product of rational matrices along axes of a lattice; the projective limit of the partial products encodes the constant. Rank-6 fields correspond to ${}_6F_5$ hypergeometric data and are a current frontier of the search.

The computational kernel is deceptively small — a chain of 6×6 matrix multiplies — but has two properties that make it an HPC problem in miniature:

- **Explosive growth.** With the corrected Ramanujan Dreams semantics at $z = 1/2$, each unit-axis operator is a primitive integer matrix bounded by 17 bits, yet the *product* of 1000 such operators is a 9546-bit integer matrix. Exact evaluation therefore pays a growing big-integer cost, and IEEE double precision cannot represent the entries — only their projective ratio, and only if renormalised.
- **Massive independent parallelism.** Discovery campaigns evaluate these walks by the million with different shifts/directions. This is embarrassingly parallel across trajectories, which is exactly the regime a laptop GPU (Apple Metal) is built for — provided the arithmetic can be made either exact (RNS) or acceptably precise (df64).

Researchers building on this engine repeatedly face the same practical questions: *How much faster is the approximate path, and how much precision does it cost? Is an RNS decomposition competitive with FLINT at this matrix size? When does the GPU actually win?* We answer them with an artefact that a third party can re-run and re-verify, rather than with quoted numbers.

Contributions.

1. Three deterministic, SHA-256-pinned ${}_6F_5$ operator corpora (`sparse_x1`, `x6`, `dense_pos`) and exact references, generated directly from the repository's `lib/cmf_walk_corrected.py` oracle, spanning 17- to 62-bit operator entries.
2. A uniform timing harness (warm-ups, repeat medians with MAD) covering exact and approximate CPU and GPU back-ends, with mandatory digest verification for exact methods and projective-error scoring for floating-point methods.

3. An exact RNS engine (32-bit Montgomery multiplication + incremental CRT) in both threaded-CPU and Metal-GPU form, with an explicit modulus-capacity certificate, and a *measured* end-to-end (kernel + parallel CRT) exact throughput rather than a kernel-only figure.
4. A **fair** CPU/GPU throughput comparison: a new CPU trajectory-parallel mode runs many independent chains across all 18 cores (GMP, FLINT, Accelerate), so GPU batch throughput is compared against saturated-CPU throughput, not single-chain latency.
5. A batch-scaling sweep and a bit-growth/crossover sweep (two figures) that answer *when* — at what batch size and integer size — RNS or the GPU becomes preferable.
6. Full results on an Apple M5 Max, released with code, data, and dumps under a permanent DOI (Vesterlund 2026).

1.3 2. Background

1.3.1 2.1 Projective matrix-chain limit

A CMF walk produces $P_N = \prod_{t=1}^{N} A_t$ with $A_t \in \mathbb{Z}^{6\times 6}$ (after clearing denominators to a primitive integer representative, which does not change the projective limit). Only the *direction* of P_N matters, so an approximate method is acceptable if the normalised matrix $P_N/\|P_N\|_\infty$ is recovered to sufficient precision. We measure the worst-case projective absolute and relative error against the exact reference at 100-digit `Decimal` precision.

1.3.2 2.2 Residue number systems and Montgomery arithmetic

Exact arithmetic can be parallelised by working modulo many small primes and reconstructing with the Chinese Remainder Theorem (Knuth 1997). We use K 30-bit primes; each residue channel is an independent 6×6 chain over $\mathbb{F}_p$, ideal for one GPU thread per $(prime, trajectory)$ lane. Inside each channel we keep all values in Montgomery form (Montgomery 1985), removing integer division from the inner multiply. The modulus product $M = \prod p_k$ must exceed $2\max|P_N|$ for signed reconstruction; the benchmark emits this certificate and refuses to declare correctness otherwise.

1.3.3 2.3 Double-float (df64) arithmetic

The fast approximate GPU path represents each entry as an unevaluated sum of two `float`s (hi, lo) and uses error-free transformations (Dekker 1971; Hida et al. 2001) with fused multiply-add, giving $ $48-bit effective mantissa without `double` hardware. A power-of-two renormalisation after every operator keeps the chain in range and records a shared exponent, so the projective limit is preserved.

1.4 3. Methods

1.4.1 3.1 Corpus

We use three workloads so that no conclusion depends on a single, unusually favourable operator structure (Table 2). All use $z = 1/2$ and shift $= (1, \dots, 1)$; they differ in the walk direction, which controls how many axis operators are applied per outer step and how fast the operator entries grow:

- **`sparse_x1`** — direction $(1, 0, \dots, 0)$, 1000 outer steps, one unit-axis operator per step; operator entries stay within 17 bits.
- **`x6`** — direction $(1, 1, 1, 1, 1, 1, 0, \dots)$, 167 outer steps (1002 operators); operator entries reach 45 bits (still `df64`- and `double`-input-exact, but not `float32`-exact).
- **`dense_pos`** — direction $(1, \dots, 1)$, 70 outer steps (770 operators); operator entries reach 62 bits, so the operators themselves *exceed* `df64` and `double` input precision — a case where approximate paths are not even input-exact.

Each rational axis operator A is converted to a primitive integer matrix (clear denominators, divide by the gcd, fix sign), preserving the projective limit while giving every back-end the identical integer workload. Every corpus and reference is content-addressed by SHA-256.

1.4.2 3.2 Back-ends

Exact: Python `int`, SymPy (Meurer et al. 2017), python-flint and native FLINT `fmpz_mat_mul` (The FLINT Team 2024), GMP `mpz_class` with `mpz_addmul` (Free Software Foundation 2024), Boost.Multiprecision `cpp_int` (Maddock and Kormanyos 2024), threaded Montgomery-RNS CPU, and a Metal RNS GPU kernel. Approximate: Apple Accelerate `cblas_dgemm` (Apple Inc. 2024), NumPy (Harris et al. 2020), and a Metal df64 kernel. All native code is built `-O3`; Metal is compiled with fast-math disabled so the df64 error-free transforms are honoured.

1.4.3 3.3 Timing methodology and throughput definitions

Each back-end runs 2 warm-ups then 7 timed repeats; we report the median and median absolute deviation (MAD). GPU kernels report both `MTLCommandBuffer` GPU-native time and host wall time. We are careful to compare *comparable* quantities and define three throughput measures explicitly:

- **Single-chain latency** — time for one chain to one exact/approximate result (Tables 3–4). This is a *latency*, not a throughput.
- **Batch-replay throughput** — batch B replays the *same* chain across lanes. For the GPU this saturates the kernel; for a fair CPU counterpart we added a **trajectory-parallel** mode that runs B independent chains across all 18 cores (one chain per worker, atomic work-stealing) for GMP, FLINT, and Accelerate. Both replay the same chain, so GPU and CPU throughput are directly comparable. RNS batch-replay measures the **modular kernel only** (residues).
- **End-to-end exact throughput** — for the RNS path we additionally measure the cost of reconstructing the exact integer matrix by centered CRT, run in parallel across the 18 cores. Because the modular kernel is cheaper than reconstruction, this is the true "time-to-exact-result" throughput ceiling of the RNS pipeline. GMP/FLINT produce the exact integer directly, so their trajectory-parallel throughput is already end-to-end.

We also report **unique-trajectory** application throughput from the production df64 driver on distinct deterministic trajectories (Table 7). Two sweeps drive the figures: a batch-scaling sweep over $B \in \{1, \dots, 256\}$ (Figure 1) and a bit-growth sweep over chain length $N \leq 2000$ (Figure 2).

1.4.4 3.4 Hardware and software

See Table 1. Timings for a given workload come from a single `run_all.py` invocation; the batch-scaling sweep is a separate continuous run (so its CPU points reflect a GPU-warmed thermal state) and GPU throughput headline numbers were re-measured cold to avoid thermal bias. Provenance (chip, cores, compiler, BLAS, git commit) is captured in `system_metadata.json`.

Table 1. Test platform.

Component	Value
Chip	Apple M5 Max (`Mac17,6`)
CPU	18-core: 6 super cores + 12 performance cores (Apple Inc. 2025)
GPU	40-core, Metal 4
Memory	128 GB unified
OS	macOS 26.4, Darwin 25.4.0 (arm64)
Compiler	Apple clang (Xcode 21)
Build	CMake, `-O3 -DNDEBUG`
Libraries	FLINT 3.x, GMP 6.3, Boost 1.9x, NumPy 2.2 (Accelerate BLAS), SymPy 1.14, gmpy2 2.3

1.5 4. Results

1.5.1 4.1 Correctness across three workloads

Table 2. Corpus statistics for the three workloads. "Input-exact" lists the approximate input formats that represent every operator exactly.

Workload	Operators	Max op. (bits)	Reference (bits)	RNS K	Modulus (bits)	Input-exact
`sparse_x1`	1000	17	9546	319	9570	float32, df64, double
`x6`	1002	45	9179	306	9180	df64, double
`dense_pos`	770	62	445	15	450	none

For **every** workload, all exact back-ends (GMP, FLINT, Boost, RNS-CPU, RNS-GPU) reproduced the exact reference digest (`correct: true`) and the RNS engines certified `capacity_sufficient: true`. The `sparse_x1` corpus/reference digests are `6734cfeb…` / `19552ce2…9071e`. This holds even for `dense_pos`, whose 62-bit operators exceed `double`/`df64` input precision — there the approximate paths are wrong at the *input* stage (Section 4.4), while the exact paths remain correct.

1.5.2 4.2 Single-chain latency (`sparse_x1`)

Table 3. Exact single-chain latency (`sparse_x1`, N=1000, median of 7–9 on a quiet machine). Lower is better.

Back-end	Median (ms)	MAD (ms)	Notes
GMP `mpz_class`	**8.48**	0.05	fastest exact
FLINT native, 1 thread	11.95	0.08	
python-flint	15.17	—	
RNS-CPU, 18 threads (kernel)	15.74	—	+ 22.9 ms CRT → 38.6 ms exact
Boost `cpp_int`	28.95	0.13	
gmpy2	79.30	—	
SymPy	85.18	—	
Python `int`	123.28	—	

Table 4. Approximate back-ends: latency and projective error (`sparse_x1`).

Back-end	Median (ms)	Proj. rel err	Exact?
Accelerate `dgemm`	**0.266**	2.4e-15	no
NumPy (Accelerate BLAS)	11.01	2.4e-15	no
Metal df64, batch 1	11.72 (GPU)	2.6e-14	no

Accelerate is $ 32×$ faster than the fastest exact method (GMP) at a projective relative error of 2.4×10^{-15}. Constant-recognition pipelines screen the projective direction at tens of significant digits before an exact re-check, so this error is comfortably within typical screening tolerances; we do not claim a specific recovery rate here.

1.5.3 4.3 Throughput: the fair comparison

RNS batch-replay measures the **modular kernel only**. To obtain the exact integer per trajectory one must reconstruct by CRT; a single centered-CRT reconstruction of the 9546-bit result costs $ $23 ms on one core (embarrassingly parallel across trajectories). Table 5 therefore separates modular-kernel throughput from end-to-end exact throughput, and compares the GPU against the CPU running *trajectory-parallel* across all 18 cores.

Table 5. `sparse_x1` throughput (K=319), peak/quiet-machine. "modular" = residues only; "exact" = usable integer result; "approx" = projective.

Path	Arithmetic	Throughput (traj/s)
RNS-GPU modular kernel, batch 26	exact (modular)	2975
CPU GMP, trajectory-parallel (18 cores)	**exact (end-to-end)**	**1468**
RNS end-to-end (kernel + parallel CRT, 18 cores)	exact (end-to-end)	524
df64-GPU, batch 26	approx	2507
CPU Accelerate, trajectory-parallel (18 cores)	approx	$ $16500

The key finding: **for a result that is exact as an integer, trajectory-parallel CPU GMP (1468/s) beats the RNS-GPU end-to-end path (524/s)**, because host CRT, not the modular kernel, is the bottleneck. The RNS-GPU modular kernel is fast (2975/s) but only produces residues; it pays off only when reconstruction is amortised (few hits) or the downstream search can operate in residue space. For *approximate* screening the 18-core CPU (Accelerate, $ $16500/s) is far ahead of the df64 GPU at this matrix size.

Figure 1 shows how these throughputs scale with batch size B (sustained, continuous-sweep timing). The df64 GPU only overtakes the CPU near $B = 256$; the RNS-GPU modular kernel saturates around $B = 64$; the exact end-to-end RNS path is CRT-bound and flat.

1.5.4 4.4 Robustness across workloads

Table 6. Cross-workload summary (per-workload run, median of 7). Latency in ms; GPU throughput in traj/s (batch $\approx K$-saturating, re-measured cold); df64 projective relative error.

Workload	GMP	FLINT	RNS-CPU 18t (+CRT)	Accel.	RNS-GPU	df64-GPU	df64 rel err
`sparse_x1`	8.5	11.9	15.7 (+22.9)	0.27	2975	2507	2.6e-14
`x6`	10.1	46.9	49.5 (+92)	0.40	2778	2583	3.0e-14
`dense_pos`	2.5	2.3	1.6 (+0.08)	0.25	5509	4681	**2.18**

The regime structure is stable across operator densities. The exception is scientifically important: on `dense_pos` the 62-bit operators cannot be represented exactly in `double`/`df64`, so the approximate GPU/BLAS paths are wrong *before any multiplication* (projective relative error 2.18 for df64, 5.2 for Accelerate) while every exact path still matches the reference. This is a concrete case where the fast filter is inapplicable and exact arithmetic is mandatory.

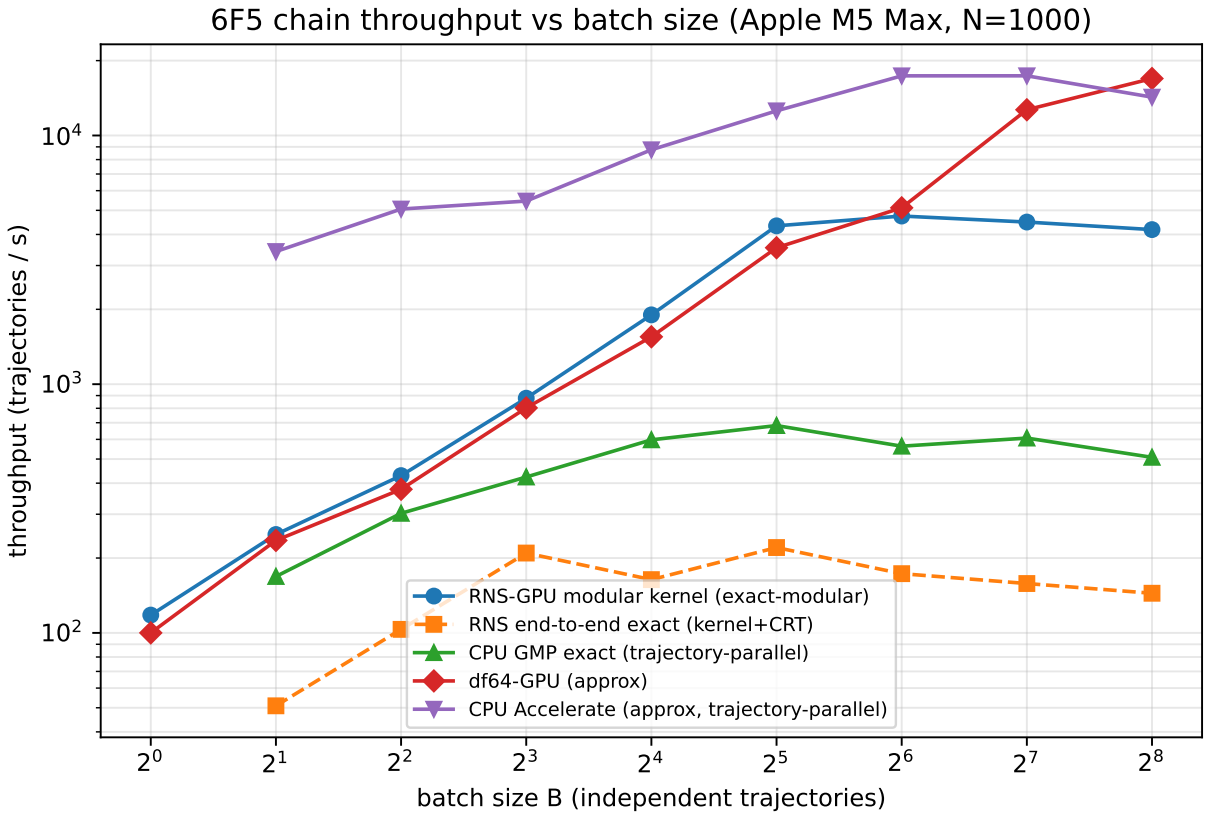


Figure 1: **Figure 1.** Sustained 6F5 chain throughput versus batch size B (`sparse_x1`, N=1000, log–log). Exact-modular (RNS-GPU kernel) and exact end-to-end (CPU GMP; RNS kernel+CRT) are distinguished from approximate (df64-GPU; CPU Accelerate). Measured in one continuous sweep, so CPU points reflect a GPU-warmed thermal state.

1.5.5 4.5 Bit growth and the exact-arithmetic crossover

Table 7. Bit growth and single-result exact times vs chain length N (`sparse_x1`).

N	bits(P_N)	K	GMP (ms)	FLINT (ms)	RNS end-to-end (ms)
100	638	22	0.33	0.27	1.8
500	4283	143	3.26	2.56	52
1000	9546	319	10.35	14.86	478
2000	21071	703	94.3	77.7	2044

The largest entry grows $ $9.5 bits per operator, so K (hence GPU lane count and CRT cost) grows linearly in N (Figure 2). On a *single result*, GMP remains the fastest exact method throughout the range; the RNS single-result time is dominated by the growing CRT. The RNS/GPU advantage is therefore a *throughput* advantage on the modular kernel, not a single-result-latency advantage, and it is progressively eroded on the exact end-to-end path as K grows — which motivates GPU-side reconstruction for long walks.

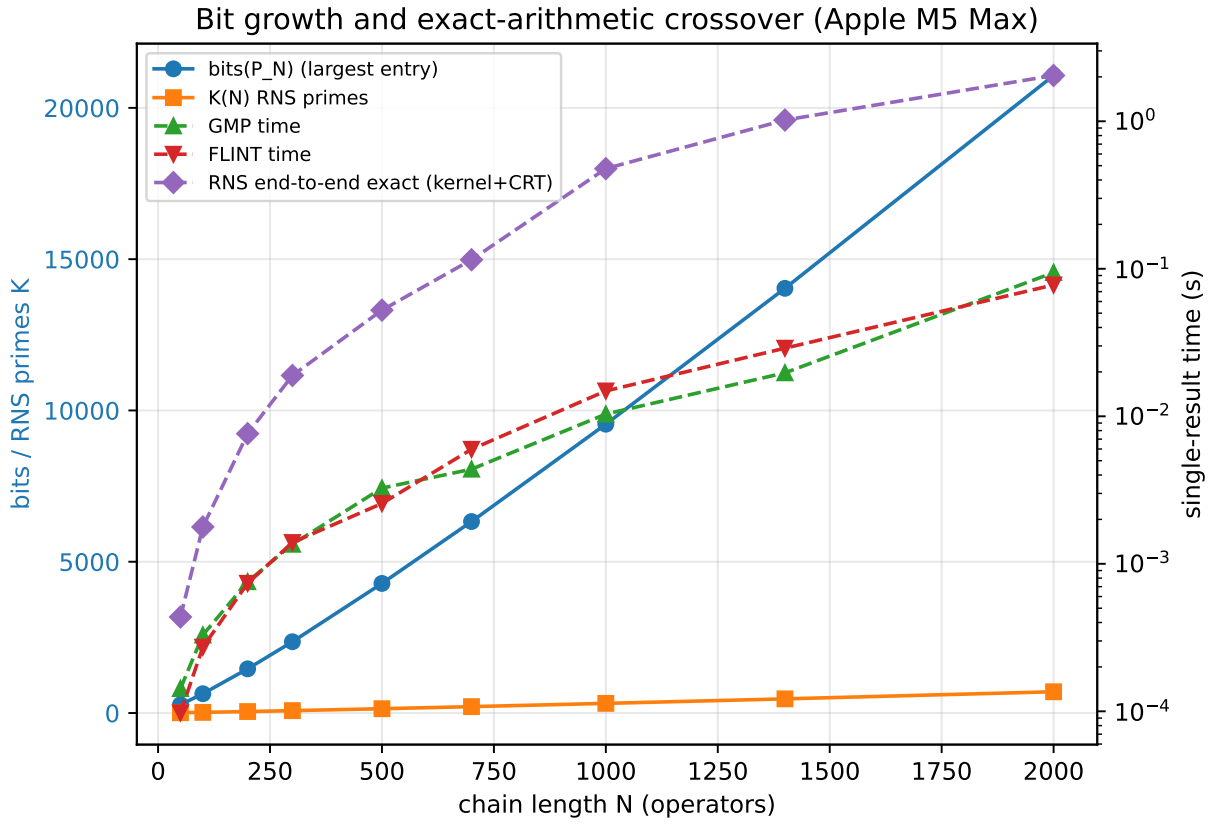


Figure 2: **Figure 2.** Bit growth of the exact product and the number of RNS primes K (left axis) with single-result exact times (right axis, log) versus chain length N (`sparse_x1`). GMP and FLINT track each other; the RNS end-to-end time is dominated by CRT and grows fastest.

1.5.6 4.6 Unique-trajectory production throughput

Table 8. Production df64 walk — unique trajectories (20000 traj, 1000 steps).

Metric	Value
Throughput (median of clean repeats)	$ $2849 traj/s
Range (excl. one thermally throttled repeat)	2773–3144 traj/s
Rank-6 finite (non-flagged) / total	6011 / 20000 (30.1%)
Timing basis	host-reported; GPU-native via `production_timing_patch.diff`

The 30.1% non-flagged fraction is an aggregate: the walk oracle flags a trajectory as either a genuine CMF singularity or a numerically ill-conditioned step, and the production driver records only the aggregate flag count. Separating true singularities from conditioning flags requires per-trajectory status codes and is left to future work; we therefore report the flag rate descriptively rather than attributing it wholly to the search space.

1.6 5. Discussion

The central question this benchmark answers is not "is the GPU faster" but *at what arithmetic size and batch parallelism does RNS (and the GPU) become preferable to multiprecision arithmetic for exact small-matrix chains?* The data give a three-part answer.

Single result: GMP wins. For one chain, big-integer GMP is the fastest exact method (8.5 ms), ahead of FLINT (11.9 ms) and Boost (29 ms); FLINT threading gives no benefit because a 6×6 multiply is far below the parallel-work threshold. Accelerate `dgemm` is fastest overall (0.27 ms) but only projectively correct. Both GPU paths lose on a single chain: a Metal launch (and, for RNS, a $ $23 ms host CRT) cannot compete with an 8.5 ms CPU big-integer product.

Exact throughput: the CPU is competitive, and modular≠end-to-end. This is where the fair comparison matters most. The RNS-GPU *modular kernel* is fast (2975 residue-trajectories/GPU-s), but a discovery campaign that needs the exact integer of each trajectory must reconstruct by CRT, and CRT — not the kernel — is the bottleneck: the end-to-end exact RNS path sustains only $ $524 traj/s, *below* trajectory-parallel CPU GMP at $ $1468 traj/s. So the RNS-GPU wins on exact work only when reconstruction is amortised (e.g. one only reconstructs the rare hits, or compares/filters directly in residue space). Reporting the modular kernel alone as "exact GPU throughput" would overstate the exact end-to-end advantage by roughly 6×.

Approximate throughput: GPU only wins at large batch. For df64 screening the 40-core GPU reaches parity with the 18-core Accelerate CPU ($ $16.5k traj/s) only near $B = 256$ (Figure 1); at this matrix size the CPU dominates until the GPU is fully saturated. The fast path is nonetheless a legitimate *filter* on input-exact workloads (`sparse_x1`, `x6`): df64 holds projective error to $\sim 10^{-14}$, so millions of trajectories can be screened approximately and the handful of hits re-checked exactly. The `dense_pos` case is the cautionary counter-example: when operators exceed 53 bits the approximate paths are wrong at the input, and only exact arithmetic is admissible.

RNS capacity is *a posteriori* here. The certificate $K = 319$ (9570-bit modulus) is verified against the *known* exact reference: $\prod_k p_k > 2 \max |P_N|$. For an *unknown* trajectory this is not available in advance; one must instead size K from an *a priori* bound — the conservative product bound (17444 bits for `sparse_x1`, i.e. $K \approx 582$), which roughly doubles the lane count and CRT cost — or use adaptive CRT with an extra verification prime to detect insufficient capacity. Because the reference grows $ $9.5 bits per operator, K (and hence lane count and CRT cost) grows linearly in N (Figure 2), making GPU-side reconstruction the natural next step for long walks.

1.7 6. Reproducibility

The complete artefact — source, corpora, per-workload results, sweep outputs, and figures — is archived at DOI 10.5281/zenodo.22045418 and mirrored at `https://github.com/VesterlundCoder/applesilicon_rns_benchmark` (Vesterlund 2026). The three workloads are regenerated by:

```
python run_all.py --repo-root <repo> --label M5-Max --preset sparse_x1  --outer-steps 1000 --repeats 7
python run_all.py --repo-root <repo> --label M5-Max-x6 --preset x6 --outer-steps 167 --skip-production
python run_all.py --repo-root <repo> --label M5-Max-dense --preset dense_pos --outer-steps 70 --skip-pr
```

and the two figures by `sweep_batch.py`, `sweep_bitgrowth.py`, and `make_figures.py`. Exact dumps (`dumps/*.txt`, `*.rns`) and the reference JSON allow independent re-verification via `verify_results.py` without trusting our harness. Every corpus and reference is content-addressed by SHA-256; a mismatched digest fails the run.

Environment note (Apple Silicon). The native build requires `brew install boost gmp flint`; the CMake configuration auto-discovers the Homebrew prefix. Two portability fixes are included: an explicit `long` cast for `int64_t→mpz_class` assignment (GMP has no unambiguous `long long` overload under LP64), and running the production df64 binary from the repository root so it can locate its Metal source.

1.8 7. Limitations and future work

- **Single machine.** Results are for one Apple M5 Max. The harness is label-parameterised for cross-device runs (e.g. an Apple M1 baseline, or a discrete-GPU/LUMI reference on the identical corpus); those are future work.
- **Thermal variance.** On an actively-cooled laptop, single-shot latencies and sustained throughput differ by up to $ 2–4×$ depending on thermal state. We report quiet-machine medians for headline latencies and label the batch-scaling sweep (Figure 1) as sustained; both the cold and sweep data are released so readers can see the spread.
- **Host-side CRT.** Reconstruction is currently CPU/Boost and dominates the exact end-to-end RNS path; moving CRT onto the GPU is the clearest route to making exact GPU throughput competitive with trajectory-parallel GMP.
- **A priori RNS sizing.** The capacity certificate here is *a posteriori*; a production search needs an a-priori bound or adaptive CRT with a verification prime (Section 5).
- **Per-status flagging.** The production driver records only the aggregate non-flagged fraction; a per-trajectory breakdown of singular vs. ill-conditioned flags is future work.
- **GPU-native production timing.** Production df64 throughput is host-reported; `production_timing_patch.diff` switches it to `GPUStartTime/GPUEndTime`.

1.9 Data Availability Statement

All code, the three SHA-256-pinned corpora, exact references and dumps, per-workload results, batch-scaling and bit-growth sweep outputs, system metadata, and the scripts that generate the two figures are openly available at DOI 10.5281/zenodo.22045418 and at `https://github.com/VesterlundCoder/applesilicon_rns_benchmark` (Vesterlund 2026). No third-party or restricted data were used.

1.10 Author Contributions

D.V. designed and implemented the benchmark suite, the RNS CPU/GPU and Metal df64 kernels, the trajectory-parallel CPU throughput and CRT reconstruction modes, ran all experiments, analysed the results, produced the figures, and wrote the manuscript.

1.11 Funding

This research received no external funding. All experiments were run on the author's personal hardware.

1.12 Conflict of Interest

The author declares that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

1.13 Generative AI Disclosure

An AI coding assistant was used to help implement and debug benchmark code and to copy-edit prose. The author verified all code, designed the experiments, checked every reported number against the released artefact, and takes full responsibility for the content. No generative AI was used to fabricate data or figures; all figures are generated deterministically from the released measurements.

1.14 Acknowledgements

We thank the FLINT, GMP, Boost, and NumPy communities for the exact-arithmetic baselines used here, and the Ramanujan Machine / conservative-matrix-field line of work (Raayoni et al. 2021; Elimelech et al. 2024; Weinbaum et al. 2025) that motivates the workload.

1.15 References

Apple Inc. 2024. *Accelerate Framework (BLAS/LAPACK, vecLib).*

Apple Inc. 2025. *MacBook Pro (14-Inch and 16-Inch, M5 Pro or M5 Max) — Technical Specifications.*

Dekker, T. J. 1971. "A Floating-Point Technique for Extending the Available Precision." *Numerische Mathematik* 18 (3): 224–42. https://doi.org/10.1007/BF01397083.

Elimelech, Rotem, Ofir David, Carlos De la Cruz Mengual, et al. 2024. "Algorithm-Assisted Discovery of an Intrinsic Order Among Mathematical Constants." *Proceedings of the National Academy of Sciences* 121 (25): e2321440121. https://doi.org/10.1073/pnas.2321440121.

Free Software Foundation. 2024. *The GNU Multiple Precision Arithmetic Library (GMP).*

Harris, Charles R., K. Jarrod Millman, Stéfan J. van der Walt, et al. 2020. "Array Programming with NumPy." *Nature* 585: 357–62. https://doi.org/10.1038/s41586-020-2649-2.

Hida, Yozo, Xiaoye S. Li, and David H. Bailey. 2001. "Algorithms for Quad-Double Precision Floating Point Arithmetic." *Proceedings of the 15th IEEE Symposium on Computer Arithmetic (ARITH-15)*, 155–62. https://doi.org/10.1109/ARITH.2001.930115.

Knuth, Donald E. 1997. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms.* 3rd ed. Addison-Wesley.

Maddock, John, and Christopher Kormanyos. 2024. *Boost.multiprecision.*

Meurer, Aaron, Christopher P. Smith, Mateusz Paprocki, et al. 2017. "SymPy: Symbolic Computing in Python." *PeerJ Computer Science* 3: e103. https://doi.org/10.7717/peerj-cs.103.

Montgomery, Peter L. 1985. "Modular Multiplication Without Trial Division." *Mathematics of Computation* 44 (170): 519–21. https://doi.org/10.1090/S0025-5718-1985-0777282-X.

Raayoni, Gal, Shahar Gottlieb, Yahel Manor, et al. 2021. "Generating Conjectures on Fundamental Constants with the Ramanujan Machine." *Nature* 590 (7844): 67–73. https://doi.org/10.1038/s41586-021-03229-4.

The FLINT Team. 2024. *FLINT: Fast Library for Number Theory*.

Vesterlund, David. 2026. *Apple Silicon ${}_6F_5$ RNS Benchmark: Code, Data, and Results*. https://doi.org/10.5281/zenodo.22045418.

Weinbaum, Ofir, Elyasheev Leibtag, Rotem Kalisch, Michael Shalyt, and Ido Kaminer. 2025. "On Conservative Matrix Fields: Continuous Asymptotics and Arithmetic." *arXiv Preprint arXiv:2507.08138*, ahead of print. https://doi.org/10.48550/arXiv.2507.08138.